AWS — Intelligent Banking & Payment Risk Platform

A locally verified banking-risk simulation built around explainable evidence, payment policy and auditable state changes.

Why I built it

I wanted to explore what happens between receiving a payment request and making a defensible decision about it. A risk score is not a fraud verdict. It does not, on its own, decide whether a payment should be approved, challenged, held for an analyst, declined or blocked.

I therefore separated the behavioural assessment from financial restrictions, payment policy, verification and human review. The application models synthetic customers, accounts and transactions; it does not handle real money or connect to payment rails.

Architecture options I considered

Amazon Kinesis — considered, not implemented. I considered a managed streaming service for continuous, high-throughput transaction-event ingestion and multiple downstream consumers. This version receives API submissions and assesses them synchronously. There was no measured streaming-ingestion requirement that justified adding Kinesis.

Amazon DynamoDB — considered, not implemented. DynamoDB could support a low-latency behavioural feature store, separate read model or specialised idempotency access at greater scale. PostgreSQL already provides bounded history queries, relational integrity, authoritative state and unique idempotency constraints. I chose not to split the source of truth without performance evidence.

PostgreSQL and a transactional outbox — selected. Keeping assessment evidence, payment state, the original decision and business audit records in one financial commit boundary mattered more here. EventBridge and SQS adapters provide an optional downstream delivery path; they do not authorise payments.

How the platform works

The payment simulator submits a request containing a customer, account, merchant, amount, device, channel and geographic context. The backend validates ownership and inputs, then reads the customer’s actual synthetic history from PostgreSQL. The React interface renders the returned assessment instead of calculating its own risk score.

Behavioural evidence

A 90-day window establishes customer baselines such as familiar payment amounts, merchant categories, previously observed devices, countries and UTC activity hours. Ten-minute velocity checks also consider unsuccessful attempts. Other features cover new devices, shared-device signals, travel plausibility, recent declines and authentication failures.

Versioned configurable rules produce structured signals with the observed evidence, threshold and contribution. The local model is a reproducible, fixed logistic heuristic: it is untrained and must not be described as a calibrated fraud probability.

Risk assessment and policy

The local aggregation combines capped rule contributions (70%) and model score (30%) into a 0–100 risk assessment. A separate policy engine then chooses among five initial actions: approve, step-up authentication, hold for review, decline or block. Mandatory account restrictions and insufficient funds are independent controls and cannot be disabled by a scoring rule.

For additional authentication, the simulation persists a 15-minute challenge. An analyst can record notes, escalate, approve, decline or request verification for a held payment. Each original decision remains visible when a later action leads to a different final outcome.

Authorisation and traceability

Before a successful authorisation, the service rechecks the account, customer, merchant, destination and current funds. A transaction-level lock, unique idempotency key, canonical payload hash and conditional balance update protect against repeated or concurrent debits. Final state transitions cannot be overwritten.

The persisted risk snapshot includes the features, rule/model versions, explanation and customer context as it existed when the decision was made. Ordered audit records preserve the workflow, including analyst identity and actions.

Why I chose this stack

The application is a TypeScript workspace. I placed the assessment, rule evaluation, aggregation and decision policy in a pure domain package so I could test that reasoning without Express or a running database. Express and Zod handle API requests and validation, while a service layer owns lifecycle rules and Prisma/PostgreSQL manage the transactional boundary.

I used React, Vite, React Router and a typed Fetch client for the interface. Docker Compose makes the PostgreSQL-backed local workflow straightforward to run. Python tools provide an optional labelled-data logistic baseline training path and a SageMaker-compatible serving contract; the default local model remains a heuristic.

AWS integration without overstating deployment

The repository implements AWS SDK provider adapters for SageMaker inference, Bedrock explanation generation, EventBridge event publication, SQS consumption, encrypted S3 audit export, Cognito access-token verification and optional Secrets Manager/KMS configuration. Terraform describes the event bus, worker queue, dead-letter handling, private audit bucket and scoped policies.

None of those AWS services has been provisioned or live-tested as part of this repository. The local application is runnable without AWS. Private Amazon RDS for PostgreSQL is compatible with the database configuration contract but no RDS, network or production compute resources have been created.

Bedrock-generated text, if connected later, is limited to explanation. It cannot replace the evidence or change the deterministic policy decision. The transactional outbox and consumer receipts target at-least-once event delivery; the implementation does not claim exactly-once cloud delivery.

Data model and operational choices

The relational schema covers customers, accounts, transactions, merchants, devices, risk signals and assessments, immutable policy versions, challenges, analyst review, audit events and durable event receipts. Monetary fields are decimals rather than floating-point values.

PostgreSQL provides one place to query historical behaviour and commit financial state. If a larger workload eventually demonstrates a need for stream ingress or materialised low-latency features, Kinesis or DynamoDB could be introduced behind clearly defined ownership boundaries rather than added pre-emptively.

Validation and limits

The repository documents a locally tested payment lifecycle with TypeScript checks, builds, unit tests, PostgreSQL/API integration cases, smoke checks and browser interactions. Scenarios exercise approval, debiting once under idempotent replay, authentication success and failure, analyst review, account restrictions, transaction history and audit persistence. Mocked SDK contract tests cover selected AWS provider behaviour but do not prove live AWS permissions or deployment.

What the system does not claim

There are no real banking or card-network integrations, actual money transfers, double-entry ledger, refunds or settlement rails. Development identity selection and challenge buttons are simulations; production Cognito login and genuine authentication providers remain integration work.

The local scoring model is not trained or calibrated against real fraud outcomes. There is no supplied real labelled banking dataset and no demonstrated fraud-detection accuracy. The current service evaluates the lifecycle synchronously, and infrastructure-as-code has not been deployed.

What I took from building it

The most useful engineering lesson was about where authority belongs. Risk evidence informs a decision but cannot override account restrictions. An AI explanation can help an analyst understand evidence but should not quietly change policy. Downstream events communicate what happened; they are not the system of record for account balances.

Kinesis and DynamoDB remain worthwhile options for a future measured requirement, not omissions I needed to disguise. The stronger decision for this version was a consistent, inspectable payment workflow that demonstrates exactly what was assessed, decided and eventually authorised.

Where I would take it next

A next phase would involve live AWS integration checks, production identity and challenge flows, deployment controls, stronger load and failure tests, and appropriately labelled model evaluation. Streaming ingestion or a dedicated feature store should follow observed throughput, access-pattern and latency requirements.

Source repository: github.com/keenosmith-del/aws-ai-banking-and-payment-risk

Contact: keenosmith@kailor.co.za